Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	
Course Code:	2015118	Course Name:	Artificial Intelligence & Soft Computing Lab

Name of Student:	Rushikesh Subhash Khemnar
Roll No. :	81
Experiment No.:	5
Title of the Experiment:	Implement Perceptron Learning for Logic Gates (AND, OR, and XOR)
Date of Performance:	17-08-2026
Date of Submission:	31-08-2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understandin	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations	Meet Expectations	Below Expectations BE
Performance	4-5	2-3	1
Understandin	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty**:**

Signature

Date

Aim: To train a perceptron to learn AND, OR, and XOR logic gates.

Objectives:

After completing this experiment, the student will be able to:

1. To understand the concept of perceptron learning in Artificial Neural Networks.
2. To implement a single-layer perceptron for logic gate operations.
3. To train the perceptron using AND and OR gate datasets.
4. To analyze the limitations of a single-layer perceptron in learning the XOR gate.
5. To interpret the learning behaviour of perceptron models for binary classification problems. Tools Required:

- Hardware:
- Desktop/Laptop

- Software:
  - Python 3.x
  - O Visual Studio Code / PyCharm / Jupyter Notebook o NumPy Library o Windows/Linux Operating System

Dataset Details:

The experiment uses predefined truth tables of AND, OR and XOR logic gates as the training dataset. Each dataset consists of binary input combinations and their corresponding expected outputs.

Theory:

A Perceptron is the simplest form of an Artificial Neural Network and is used for solving binary classification problems. It consists of input neurons, weights, a bias, a summation unit, and an activation function that determines the output based on the weighted sum of the inputs. During training, the perceptron adjusts its weights iteratively using the perceptron learning rule to minimize classification errors.

Logic gates such as AND, OR and XOR are commonly used to demonstrate perceptron learning. The AND and OR gates are linearly separable, allowing a single-layer perceptron to learn their decision boundaries successfully. However, the XOR gate is not linearly separable, making it impossible for a single-layer perceptron to classify correctly. This limitation led to the development of multi-layer neural networks capable of solving more complex classification problems.

Procedure:

1. Prepare the truth tables for the AND, OR, and XOR logic gates.
2. Initialize the perceptron with random weights and bias.
3. Define the activation function and learning rate.
4. Train the perceptron using the AND gate dataset and observe the output. 5. Repeat the training for the OR gate dataset.
6. Train the perceptron using the XOR gate dataset and observe the results.

7. Compare the performance of the perceptron for all three logic gates and record the observations.

Description of the Experiment:

This experiment demonstrates the implementation and training of a single-layer perceptron using the truth tables of AND, OR, and XOR logic gates. The perceptron model is trained by adjusting its weights and bias based on the input data until the desired output is obtained.

The experiment illustrates the learning capability of the perceptron for linearly separable problems and highlights its inability to correctly classify the XOR gate, thereby emphasizing the need for multi-layer neural network architectures.

Detailed Description:

The perceptron model is trained by presenting binary input combinations from the logic gate truth tables and comparing the predicted output with the expected output. Based on the error obtained, the perceptron updates its weights and bias using the perceptron learning rule until convergence is achieved or the maximum number of iterations is reached.

The training process is carried out separately for the AND, OR, and XOR logic gates. Successful learning is achieved for the AND and OR gates because they are linearly separable. In contrast, the XOR gate cannot be learned using a single-layer perceptron due to its non-linear decision boundary. The results obtained from the experiment demonstrate both the capabilities and limitations of the perceptron learning algorithm.

Code:

```
import numpy as np

class Perceptron:

  def init_(self, Ir=O.1, epochs=20):
    self.lr = Ir self.epochs

    = epochs self.weights

    = None self.bias = O

  def activation(self, x):

    return 1 if x >= 0 else 0

  def fit(self, X, y):

    self.weights = np.zeros(X.shape[1])

    self.bias = O for epoch in

    range(self.epochs):
```

```
        errors = O for x, target
        in zip(X, y):
            pred = self.activation(np.dot(x, self.weights) + self.bias)
        error = target - pred self.weights += self.lr * error * x
        self.bias += self.lr * error errors += error != 0 if errors == O:
            break     def
  predict(self, X):
     return       np.array([       self.activation(np.dot(x,
       self.weights) + self.bias) for x in X


X = np.array([[0,0], [0,1], [1,0], [1,1]])
gates = {
  "AND":        np.array([O,O,O,1]),
  "OR": np.array([0,1,1,1]),
  "XOR": np.array([0,1,1,O])


for name, y in gates.items():
  model = Perceptron() model.fit(X, y) pred =
model.predict(X) accuracy = np.mean(pred == y) * 100
print("\n", name, "Gate") print("Expected . y)
print("Predicted:", pred) print("Accuracy :", accuracy,
"0/0") print("Weights , model.weights) print("Bias
model.bias) print("\nConclusion:") print("AND and OR
are successfully learned.") print("XOR cannot be learned
by a single-layer perceptron.")
```

Output:

```
Expected : [0 0 0 1]
Predicted: [0 0 0 1]
Accuracy : 100.0 %
Weights  : [0.2 0.1]
Bias     : -0.20000000000000004

 OR Gate
Expected : [0 1 1 1]
Predicted: [0 1 1 1]
Accuracy : 100.0 %
Weights  : [0.1 0.1]
Bias     : -0.1

 XOR Gate
Expected : [0 1 1 0]
Predicted: [1 1 0 0]
Accuracy : 50.0 %
Weights  : [-0.1  0. ]
Bias     : 0.0

Conclusion:
AND and OR are successfully learned.
XOR cannot be learned by a single-layer perceptron.
PS C:\Users\student\Documents\Vedant>
```

Conclusion:

A single-layer perceptron was successfully trained to learn the AND and OR logic gates, while its limitation in learning the XOR gate was observed. The experiment demonstrated the fundamental concepts of perceptron learning and the importance of multi-layer neural networks for solving non-linearly separable problems.

Questions to be Answered by Students:

1. Which logic gates were successfully learned using the single-layer perceptron?

The AND and OR logic gates were successfully learned because they are linearly separable.

2. Why was the XOR logic gate not correctly learned by the perceptron?

The XOR gate is not linearly separable. A single-layer perceptron can only create a linear decision boundary, so it cannot correctly classify all XOR inputs.

3. What role do weights and bias play in perceptron learning?

- Weights determine the importance of each input in calculating the output. ● Bias shifts the decision boundary and helps the perceptron make the correct classification.
- Both weights and bias are updated during training based on the prediction error.

4. State two observations obtained from the training of the perceptron model.

1. The perceptron successfully learned the AND and OR gates, achieving correct classification.
2. The perceptron failed to correctly learn the XOR gate because XOR is non-linearly separable.